

Design and Delivery Record

Backend Engineer Assignment

Abstract

Design and Delivery Record (DDR). This document summarizes the technical implementation of the coupon redemption API. It complements the Part A database design document by describing the API, Dockerized runtime, validation strategy, redemption safety, test strategy, and optional nice-to-have features implemented for the assignment.

1 Scope

The delivered system implements a backend service for managing and redeeming coupons. The work is organized around three areas:

- **Part A:** relational modelling and PostgreSQL schema design.
- **Part B:** API implementation for coupon listing, creation, and redemption.
- **Part C:** automated integration tests and concurrency verification.

Authentication and authorization are intentionally out of scope for this iteration. The redemption endpoint receives a `userId` in the request body so that the redemption business rules can be exercised directly.

2 Runtime Architecture

The local runtime is fully Dockerized through Docker Compose. The stack includes:

- **API:** TypeScript Fastify application running on Node.js.
- **PostgreSQL:** relational database used by the API.
- **Migration runner:** one-shot container that applies SQL migrations before the API starts.
- **Seed runner:** one-shot container that loads deterministic local development data.
- **Adminer:** browser-based PostgreSQL GUI for manual inspection.
- **Swagger UI:** browser-based OpenAPI console for exercising the API.

The startup order is explicit:

```
postgres -> migrate -> seed -> api -> swagger-ui
```

This makes the local environment reproducible with a single command:

```
docker compose up -d --build
```

3 Database Management

The schema is managed through explicit SQL migrations. The migration runner records applied migrations in `schema_migrations`, including a checksum, so already-applied migrations cannot silently drift.

The seed script loads sample campaigns, coupons, and users. The demo users use stable UUIDs only as test fixtures, allowing the Swagger examples and manual redemption flows to work after every clean bootstrap. This is not an application-level identity strategy: the database schema still defaults newly inserted primary keys to PostgreSQL-generated UUIDs.

The future campaign fixture uses an absolute September 2026 start date instead of a relative `now() + interval` expression. This keeps the “future campaign” scenario stable for reviewers who run the project days after delivery.

4 API Surface

The implemented endpoints are:

Method	Path	Purpose
GET	/health	Checks API and database availability
GET	/coupons	Lists available non-expired coupons with pagination
POST	/coupons	Creates a coupon and creates or reuses its campaign
POST	/coupons/{code}/redeem	Redeems a coupon for a user

The listing endpoint follows stable ordering:

```
campaign start timestamp ASC,  
coupon code ASC,  
coupon id ASC
```

The `id` tie-breaker makes ordering deterministic even if future schema changes weaken uniqueness assumptions.

5 Domain Decisions

5.1 Campaign Identity

An already existing campaign is identified by unique `campaigns.name`. When `POST /coupons` receives a campaign name that already exists, the API reuses the existing campaign and creates only the new coupon.

This is a simple and enforceable choice for the assignment. In a larger product, this could evolve into an external campaign identifier or a separate campaign management endpoint.

5.2 Expiration Semantics

Campaigns have a required `end_timestamp`. A campaign is expired when:

```
campaigns.end_timestamp < now()
```

Coupons have an optional `expiration_timestamp`. A coupon is expired only when:

```
expiration_timestamp IS NOT NULL  
AND expiration_timestamp < now()
```

Therefore, a null coupon expiration means no coupon-specific expiration. The coupon remains constrained by the campaign validity window.

6 Validation and SQL Safety

Input validation is implemented with Zod schemas for:

- query parameters, including bounded pagination;
- path parameters, including coupon code;
- JSON request bodies for coupon creation and redemption.

All database access uses PostgreSQL parameter binding through `pg`. User-controlled values are passed as bind parameters such as `$1`, `$2`, and `$3`; they are not concatenated into SQL strings.

Malformed JSON and validation errors are returned as client errors rather than internal server errors.

7 Redemption Correctness

Redemption is the critical operation and is implemented as a short PostgreSQL transaction. The API locks the target coupon row and its campaign row with:

```
SELECT ...  
FOR UPDATE OF coupons, campaigns
```

While those rows are locked, the service validates:

- coupon status;
- campaign status;
- campaign start and end timestamps;
- coupon expiration;
- coupon redemption limit;
- campaign redemption limit.

If validation passes, the API inserts the redemption record and increments both the coupon and campaign counters in the same transaction. If any step fails, the transaction is rolled back.

Double redemption by the same user for the same coupon is prevented in two layers:

- the service checks for an existing redemption before inserting;
- PostgreSQL enforces `UNIQUE (user_id, coupon_id)` as the final protection under races.

When many concurrent requests target the last available redemption slots, row-level locks serialize access to the relevant coupon and campaign counters. The successful transactions consume the available slots, and later transactions observe the updated counters and return a conflict response instead of exceeding the configured limits.

8 Testing Strategy

The automated test suite uses Vitest and Testcontainers. A real PostgreSQL `16-alpine` container is started for the suite. Before each test, the suite drops and recreates the `public` schema, reapplies the SQL migration, and reloads the seed data.

This gives each test an isolated database state while avoiding the overhead of starting a new PostgreSQL container for every test. The API is exercised through Fastify `app.inject()`, so tests cover routes, validation, SQL constraints, transaction behavior, and concurrency without opening a network port.

The integration tests cover:

- coupon listing, stable ordering, and null expiration semantics;
- campaign reuse by unique name;
- malformed JSON client errors;
- double redemption prevention;
- concurrent redemption requests against a limited coupon.

An additional terminal demo script creates a live concurrency scenario against the Docker Compose stack. It can simulate, for example, 50 users competing for 5 available redemption slots and prints a compact status grid plus final database counters.

9 Optional Nice-to-Have Coverage

Optional item	How it is addressed
Full Docker setup	Implemented with Docker Compose for API, PostgreSQL, migration runner, seed runner, Adminer, and Swagger UI.
Seed script or sample data generator	Implemented through an idempotent SQL seed script with stable local demo users and representative campaign/coupon states.
Structured logging	Implemented through Fastify's logger in non-test environments. Logs are structured JSON by default and disabled during tests for readable output.
OpenAPI / Swagger spec	Implemented with <code>docs/openapi.yaml</code> and a Dockerized Swagger UI console.
Input validation layer	Implemented with Zod schemas for query parameters, path parameters, and request bodies.
Rate limiting or abuse controls	No dedicated rate limiter is implemented in this assignment. Basic abuse controls include bounded pagination, strict validation, SQL parameter binding, and transactional limit enforcement. A production version should add a Fastify rate-limit plugin or gateway-level throttling.

10 Known Trade-Offs

- Campaigns are identified by name for assignment simplicity; an external campaign id would be preferable in a larger product.
- Authentication is not implemented because it is outside the assignment scope.
- Rate limiting is documented as a production hardening step rather than implemented application behavior.
- The OpenAPI specification is maintained as a static contract; in a larger codebase, schema generation could be automated from route schemas.

11 How to Review

A reviewer can validate the delivery with:

```
cp .env.example .env
docker compose up -d --build
npm test
npm run demo:concurrency:5-slots
```

The Swagger UI console is available at:

```
http://localhost:8082
```